

최현규

데이터 엔지니어링 · MLOps 포트폴리오

hyeongyu.data@gmail.com · github.com/hyeongyu-data

이전에는 이커머스 플랫폼을 1인 개발·운영했으며, 현재는 데이터 엔지니어링과 MLOps를 중심으로 역량을 확장하고 있습니다.

제가 맡아온 일

웹 서비스를 직접 배포하고 운영하면서 외부 결제·주문 API를 연결하고 사용자 행동 데이터를 수집했습니다. 최근에는 팀의 ML 실험 환경을 클라우드에 구성하고, 개인 프로젝트로 수집·정제·알림·적재 흐름을 구현하고 있습니다.

이 포트폴리오는 사용한 기술보다 **어떤 문제를 맡았고, 어떻게 확인하고 개선했는지**를 중심으로 정리했습니다.

프로젝트 한눈에 보기

프로젝트	담당 범위	확인할 수 있는 역량
Auto Research	5인 팀 인프라 전담	IaC, GitOps, 관측, 장애 대응
Air Quality Project	개인 개발	이벤트 처리, 결측·재시도, 운영 점검
Ecommerce ETL	개인 개발 · 진행 중	원천별 매핑, 품질 검사, 적재 구현
Construction Risk Agent	라우터·합성·장비비	에이전트 실행 흐름, 계산 로직

운영 경험에서 이어온 기준

배포 후 상태까지 확인합니다. 설정 파일이 맞는지뿐 아니라 실제 리소스와 실행 로그가 같은 상태를 가리키는지 확인합니다.

실패한 데이터와 정상적인 빈 결과를 구분합니다. 수집 실패가 정상 데이터처럼 다음 단계로 넘어가지 않도록 입력 제약과 예외 처리를 둡니다.

복구 과정도 코드와 문서에 남깁니다. 임시 조치로 해결한 리소스는 관리 코드에 반영하고, 재구축과 재실행 때 같은 문제가 생길 수 있는지 살펴봅니다.

경력 기반

2021년부터 웹개발 업무를 맡았으며, 최근 경력에서는 헬스·뷰티 이커머스 플랫폼을 1인 개발·운영했습니다. 15개 이상 브랜드의 주문·결제 관리와 GA4/GTM 트래킹을 구축한 경험이 있습니다. 이후 SK플래닛 데이터 엔지니어 과정 2기를 수료하고 팀 프로젝트의 클라우드 인프라를 전담했습니다.

01. Auto Research

ML 실험을 실행할 인프라 만들기

2026.06-08 · 8주 · 5인 팀 · 인프라 전담

인프라 · 애플리케이션 · Airflow

프로젝트와 담당 범위

가설을 입력하면 AI 에이전트가 Kubernetes에서 ML 실험을 실행하고 리포트를 생성하는 팀 프로젝트입니다. 저는 GCP 리소스, CI 인증, 배포, 관측 환경을 담당했습니다. 모델·애플리케이션과 Airflow 업무 로직은 팀의 다른 담당 영역입니다.

Terraform · GitHub Actions · WIF

↓ 인프라 구성 및 배포 권한

GCP · GKE · Cloud SQL · Artifact Registry

↓ 실행 상태 관측

Prometheus / Grafana · ELK

초기 환경 구축

실험 코드가 실행될 개발 환경부터 구성해야 했습니다. VPC·GKE·Cloud SQL·Artifact Registry와 CI 인증·저장소 기반을 Terraform으로 관리했습니다. 초기 리소스 25개를 1주일 내 적용했고, 당시 terraform plan 의 No changes 로 코드와 배포 상태가 일치함을 확인했습니다.

관측 공백 해소

GKE·Airflow·MLflow·서빙의 상태를 함께 확인할 수 있도록 관측 작업을 8개 이슈로 나눴습니다. Prometheus/Grafana와 ELK를 1.5일 내 배포·검증하고, 실제 런타임 메트릭과 로그에 맞춰 수집 설정과 대시보드를 조정했습니다.

배포 절차와 보안

- Terraform apply 진입점을 2개에서 1개로 통합하고 8개 root를 한 번의 승인으로 적용하도록 정리했습니다.
- 서비스 계정과 WIF 권한은 분리하고 OAuth allowlist를 Kubernetes Secret으로 이관했습니다.
- Claude Code PR 리뷰에 IAM·시크릿·배포·롤백·비용 항목을 포함했습니다. 리뷰 의견에는 실행 결과와 코드 근거로 답변하도록 했습니다.

확인 범위 — 교육과정의 개발 환경에서 구축·운영한 경험입니다. 초기 배포 시점의 상태 확인을 지속적인 무장애 운영 성과로 표현하지 않았습니다.

01. Auto Research

장애에서 확인한 코드와 실행 상태의 차이

사례 A · Airflow crash-loop

증상 — Airflow가 45시간 동안 정상 실행되지 못하고 재시작을 반복했습니다.

원인 — 오래된 로컬 `tfvars` 가 잘못된 Cloud SQL 대역을 NetworkPolicy에 적용했습니다. 저장소의 설정과 실제 적용에 사용된 값이 달랐습니다.

대응 — 연결 대상과 적용된 네트워크 정책을 대조해 원인을 좁혔습니다. 권한 변수 누락 등 유사한 설정 사고 3건도 분석해, 설정과 적용 상태의 불일치를 재발 방지 과제로 정리했습니다.

배운 점 — 장애 시 애플리케이션 로그만 보는 것으로 충분하지 않았습니다. 어떤 입력값이 어떤 리소스에 적용됐는지까지 확인해야 했습니다. 45시간은 장애 지속 시간이며, 복구에 걸린 시간이나 단축 성과를 뜻하지 않습니다.

사례 B · 환경 재구축 후 admission 403

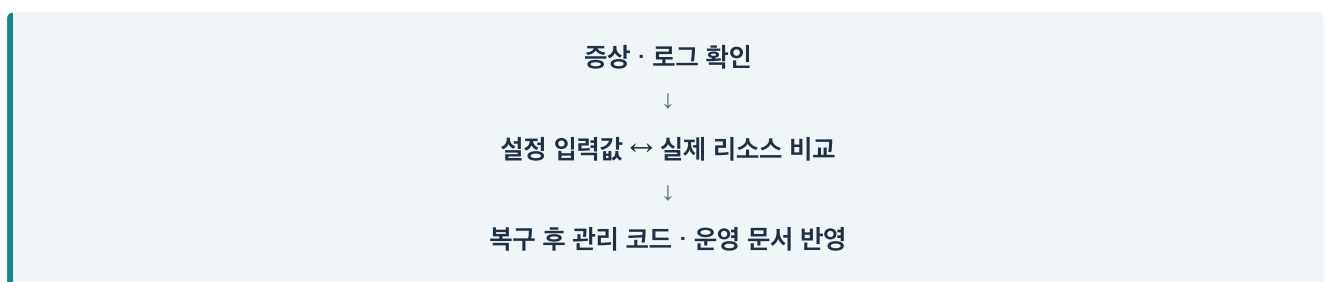
증상 — 환경을 다시 만든 뒤 배치 실행이 admission 403으로 실패했습니다.

원인 — 이전 환경에서 수동으로 생성한 Kubernetes Service Account가 재구축 과정에 포함되지 않았습니다.

대응 — 해당 계정을 Terraform 관리에 편입했습니다. 복구 과정에서 먼저 만든 리소스는 state로 가져와 실제 환경과 관리 코드가 중복되지 않도록 정리했습니다.

개선 — 다음 환경 구성 시 필요한 계정을 코드로 함께 생성할 수 있게 했습니다. 임시 복구에서 끝내지 않고 재구축 절차에 반영한 사례입니다.

이후 작업에서 적용하는 점검 순서



문제가 발생한 지점과 변경 이력을 먼저 확인하고, 코드·권한·네트워크·실행 상태를 연결해 원인을 설명하려고 합니다. 복구가 끝나면 같은 조건에서 다시 배포할 때도 필요한 설정이 유지되는지 점검합니다.

02. Air Quality Project

데이터가 없거나 처리가 실패해도 구분할 수 있게

2026.04-09 · 개인 프로젝트 · 로컬 Docker 환경

저장소 및 실행 문서

기상청·에어코리아 데이터를 하루 4회 수집해 상태를 분류하고 알림을 보내는 파이프라인입니다. 데이터 수집부터 저장·알림까지의 실패 상황을 직접 다뤄보기 위해 구성했습니다.

Airflow → 기상청 · 에어코리아 API

↓ 수집 이벤트

Kafka → 소비자 · 상태 분류

↓ 처리 결과

OpenSearch · 카카오톡 / 이메일 / Slack

결측값을 정상 수치로 만들지 않기

결측값을 0으로 바꾸면 대기질이 좋은 것으로 오해할 수 있습니다. 값이 없으면 UNKNOWN으로 분류하고 화면과 알림에서는 정보가 없음을 표시합니다. 미세먼지 두 지표가 모두 없을 때는 외부 알림을 억제하되 처리 이력을 남깁니다.

실패 후 다시 처리할 수 있게 만들기

- 결정적인 event ID를 문서 키로 사용해 OpenSearch에 upsert합니다.
- Kafka는 처리 완료 후 수동 커밋하고, 잘못된 메시지는 DLQ와 스키마 계약으로 다룹니다.
- 상태 변화에 따라 알림을 보내며, 전송 성공 시에만 중복 억제 상태를 기록합니다. 모든 채널이 실패하면 다음 처리에서 재시도할 수 있게 했습니다.
- 재연결 대기과 소비자 종료 처리를 구현하고, 시크릿이 로그에 노출되지 않도록 마스킹했습니다.

검증과 기술 선택

로컬에서 실제 데이터 수집·알림 수신을 확인했습니다. 2026년 9월 15일 회귀 테스트 248개 통과를 확인했으며, CI에서 ruff·pip-audit·gitleaks·hadolint 등을 실행하도록 했습니다.

하루 4회 수집에는 Kafka·OpenSearch가 필수적이지 않습니다. 이 프로젝트에서는 이벤트 처리와 검색·관측 구조를 학습하기 위해 사용했고, 더 단순한 대안과의 차이를 ADR에 기록했습니다. 로컬 검증 결과를 대규모 트래픽 처리 성과로 해석하지 않습니다.

03. Ecommerce ETL

서로 다른 주문 데이터를 같은 계약으로 다루기

2026.09-진행 중 · 개인 프로젝트 · 2026.09.15 기준

저장소 및 스키마 문서

이커머스에서 결제·오픈마켓 데이터를 통합 조회한 경험을 바탕으로, 수집·정제·적재 과정을 재현 가능한 개인 프로젝트로 구현하고 있습니다. 원천 데이터는 합성 데이터와 로컬 mock을 사용합니다.

합성 PG CSV · mock 오픈마켓 HTTP · GA4 mock 보고서

↓ 원천별 Airflow DAG 3개

18개 컬럼 통합 스키마 · 품질 검사 · Parquet

↓ 별도 수동 실행 DAG

DuckDB 날짜별 적재 모듈

수집·정제 구현

- PG·오픈마켓·GA4 입력을 원천별 DAG로 분리했습니다. 오픈마켓은 로컬 HTTP API를 호출하며 타임아웃과 재시도를 처리합니다.
- 원천별 필드를 통합 주문 스키마로 매핑하고 품질 검사 후 날짜별 Parquet staging에 저장합니다.
- GA4 mock 보고서는 헤더·스키마·날짜·금액·중복 항목을 확인합니다. 정상적인 0행 결과와 손상된 보고서·수집 실패를 구분합니다.

DuckDB 적재 구현

세 원천의 staging을 읽고 공통 품질검사 후 DuckDB에 적재하는 수동 실행 DAG를 구현했습니다. 트랜잭션으로 대상 날짜를 삭제·삽입하며, 정상 입력을 같은 날짜에 재실행했을 때 중복 없이 유지되는지 확인했습니다. BigQuery는 설계 목표로 남겨두었습니다.

검증 범위와 후속 작업

구분	현재 범위
수집 입력	합성 PG · 로컬 오픈마켓 API · GA4 mock 보고서
기존 Docker E2E	PG 46·오픈마켓 45·GA4 61 = 152행, 재실행 중복 0
현재 적재	DuckDB · 수동 통합 DAG · 공통 품질검사
후속 작업	BigQuery·실제 GA4 API 연결, 분석 대시보드

09.15 pytest는 91개 통과·1개 스킵입니다(Airflow 미설치). E2E는 기존 Docker 검증 기록입니다. 적재 함수 직접 호출의 대상일 혼입·부분 입력과 동시 실행은 추가 검증이 필요합니다.

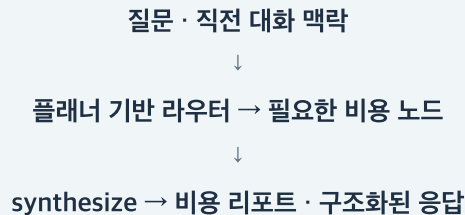
04. Construction Risk Agent

질문에 필요한 계산을 선택하고 결과를 합성하기

2026.06 · 5인 팀 · 라우터·결과 합성·장비비 에이전트 담당

포트폴리오 저장소

기상·공정 지연·자재 리스크에 따른 추가공사비를 산정하는 LangGraph 기반 팀 프로젝트입니다. 저는 질문을 어떤 노드에 전달할지 결정하는 실행 흐름, 결과 합성, 장비 대기비 계산을 맡았습니다.



고정 분기에서 동적 라우팅으로

초기 고정 A/B 분기를 플래너 기반으로 바꾸고, 질문과 이전 대화 맥락에 따라 필요한 비용 노드만 호출하도록 라우터·그래프·상태 흐름을 설계하고 구현했습니다.

여러 결과를 하나의 응답으로

`synthesize` 노드와 답변 유형별 few-shot 예시를 작성했습니다. 각 노드의 결과를 비용 리포트와 구조화된 응답으로 합성할 수 있도록 구성했습니다.

장비 대기비 계산

장비명 정규화 → 규격 매칭 → 일대여료 조회 → 대기율·일수 반영 순서로 로직을 구현했습니다. 계산에 필요한 PostgreSQL 스키마·초기 적재 데이터와 테스트를 함께 작성했습니다.

담당 기술 — Python, LangGraph, AWS Bedrock, PostgreSQL

협업 범위

백엔드 인증, 프런트엔드, 날씨·자재·인건비 노드는 다른 팀원의 담당 영역입니다. 제 기여는 공통 실행 흐름과 결과 합성, 장비비 계산에 집중했습니다.

함께 일할 때

문제가 생기면 증상과 원인을 구분하고, 변경 이유를 코드와 문서에 남깁니다. 웹 서비스 운영 경험과 인프라 프로젝트 경험을 바탕으로, 데이터 파이프라인의 실행 상태를 확인하고 팀이 문제를 추적할 수 있는 환경을 만드는 역할을 맡고 싶습니다.